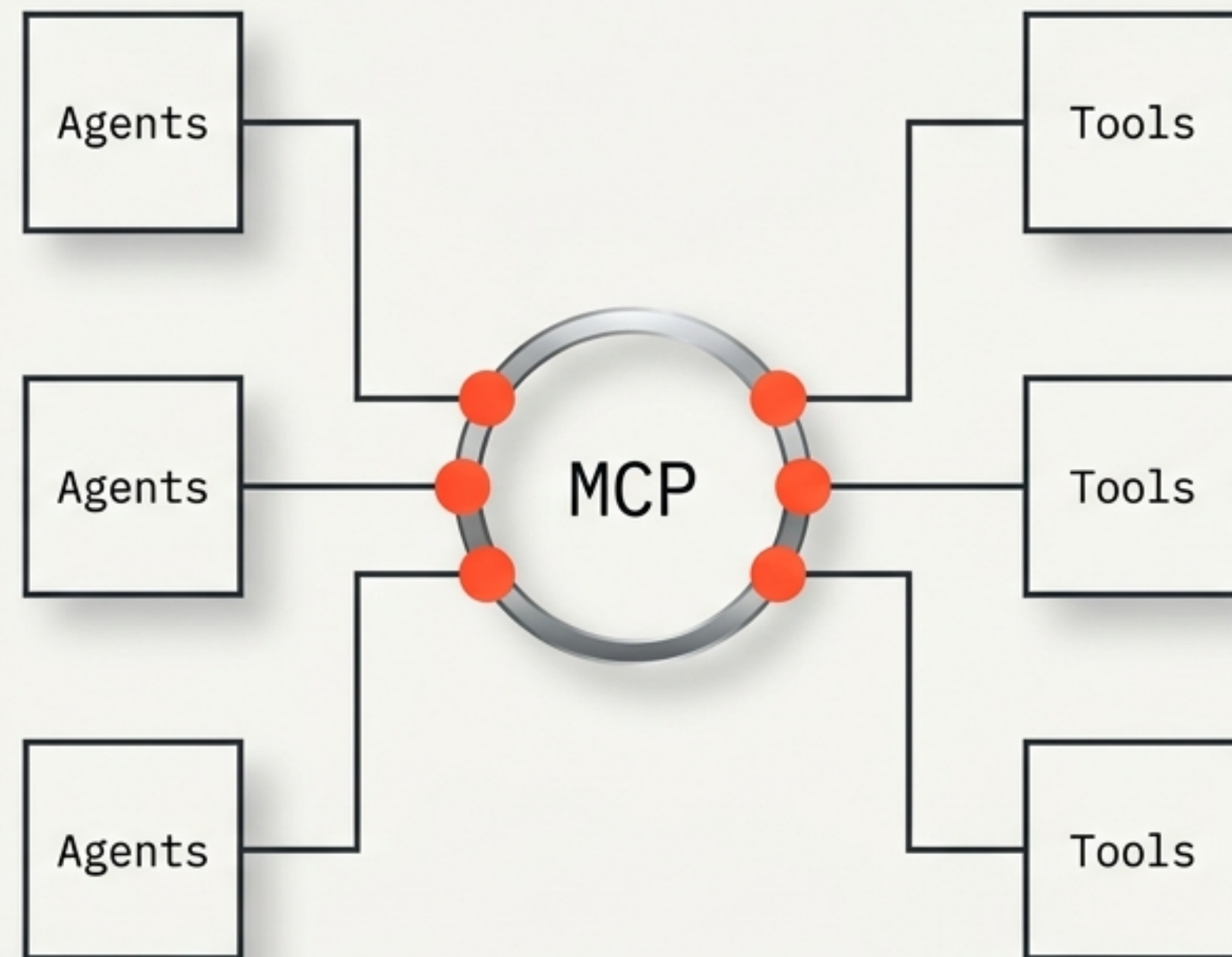


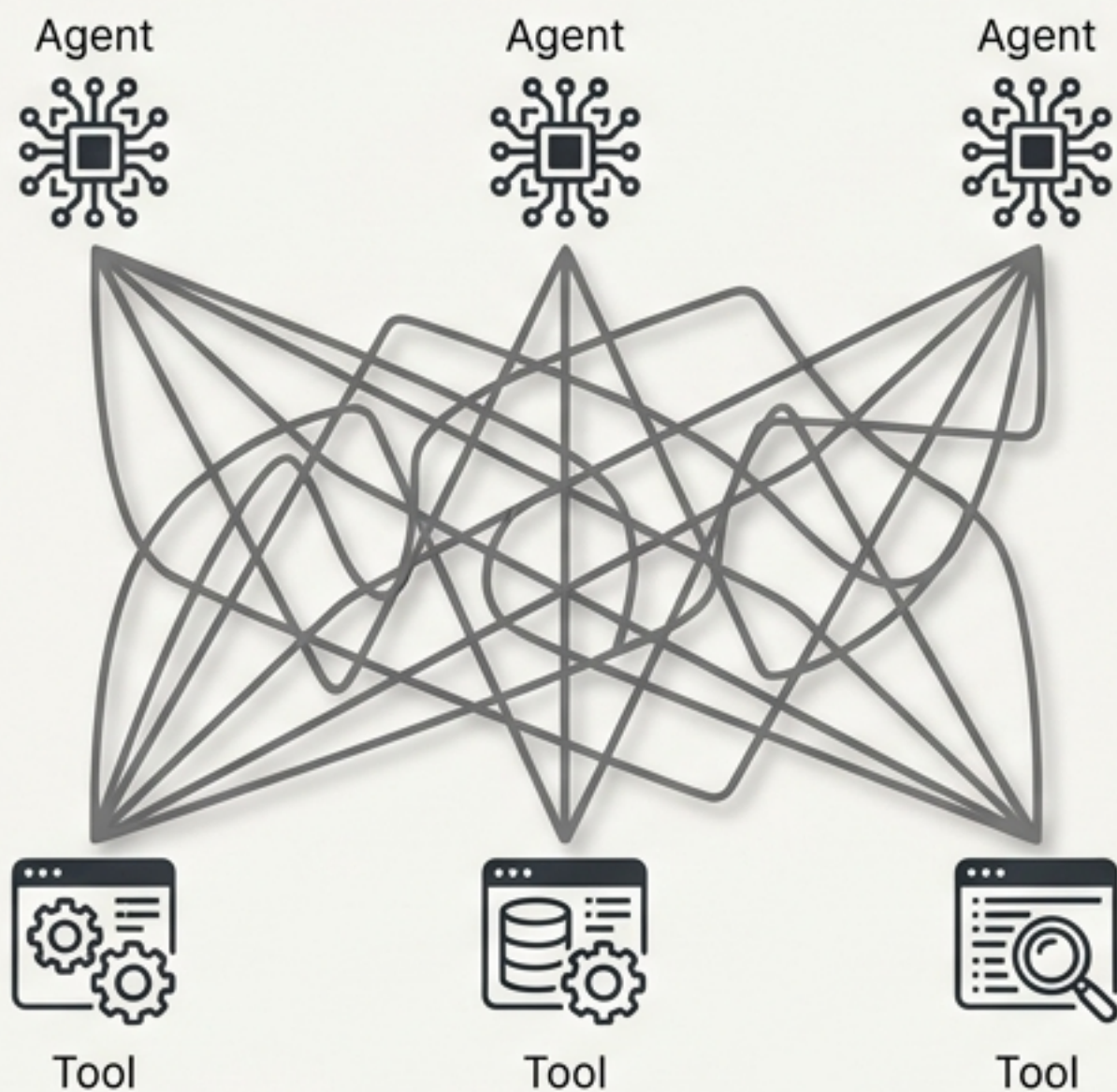
Introduction to the Model Context Protocol

- **The Communication Backbone:** An open standard published by Anthropic in November 2024 that transforms how AI agents interact with the world.
- **A Protocol, Not an Agent:** MCP does not change how agents reason; it dictates how they securely connect to external tools and data.
- **The HTTP of Agentic AI:** Just as HTTP gave every device a common language for the web, MCP gives agents a standardised language for capability servers.
- **A Universal Ecosystem:** Replaces bespoke, brittle tool integrations with a single, universally understood set of rules.

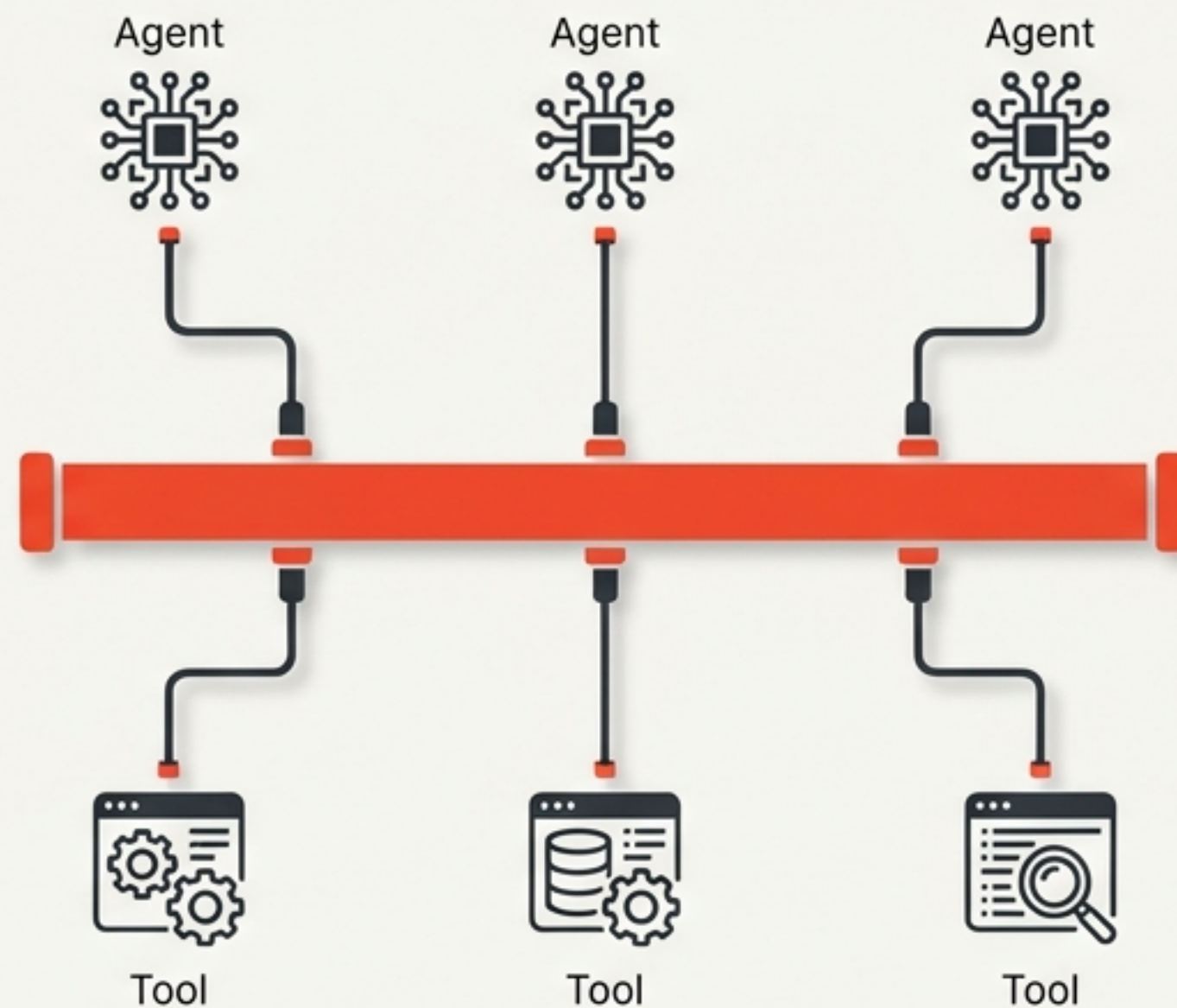


The USB-C of Agentic AI

Before MCP (The N×M Nightmare)



After MCP (The N+M Advantage)



- **The Fragmentation Problem:** Previously, developers had to build custom N×M connectors for every agent-tool pairing.
- **The Standardised Solution:** MCP acts as a universal port, allowing compatible agents and tools to connect automatically.
- **The Universal Cable:** Just as USB-C eliminated unique chargers, MCP eliminates unique tool integrations.
- **Build Once, Run Anywhere:** Build one MCP server for a tool, and all agents gain access to it instantly.

The Architecture of MCP



01. The Host

The main AI app orchestrating the workflow (e.g., Claude Desktop). Think of this as the laptop itself.



02. The Client

Lives inside the Host. Manages connections and discovers tools. Think of this as the USB-C port on the laptop.

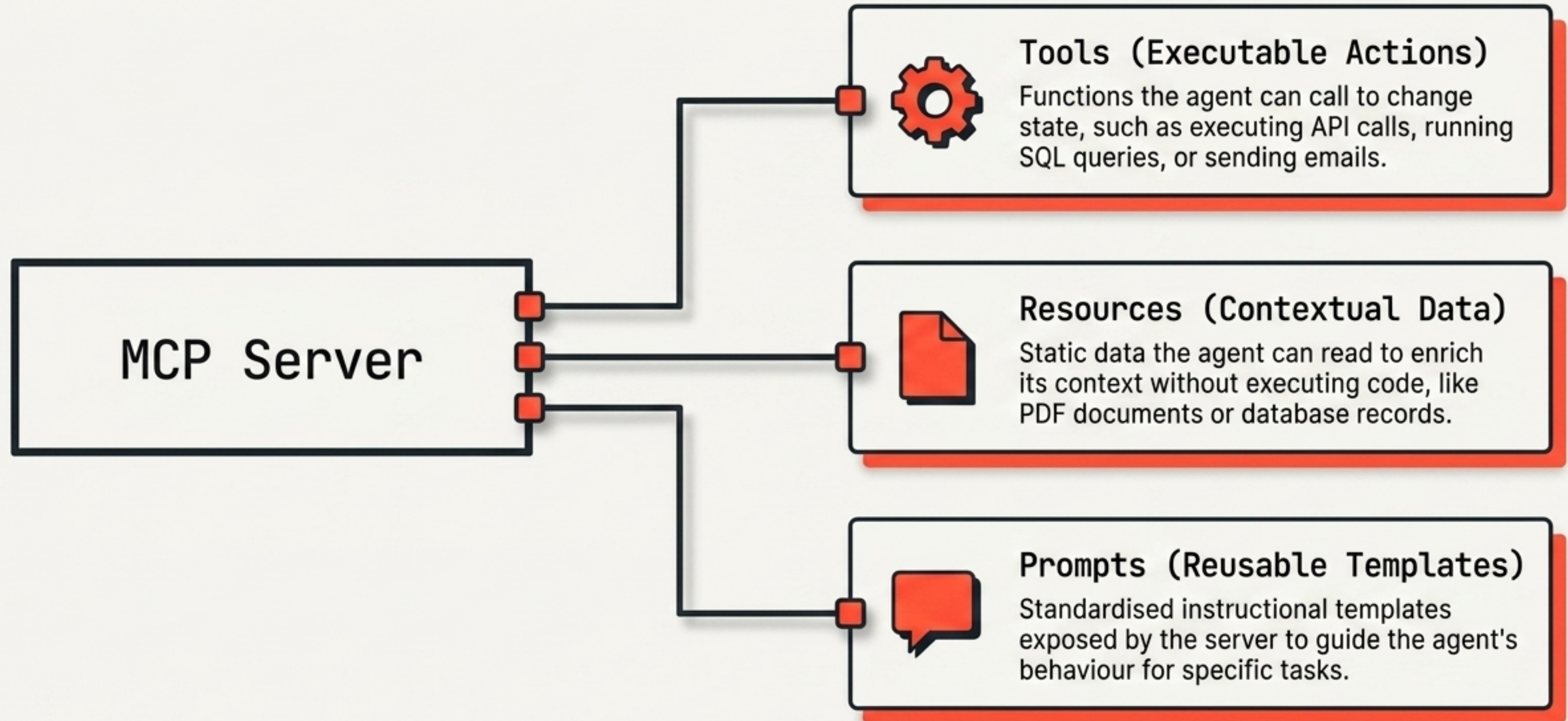


03. The MCP Server

A standalone process exposing capabilities. Think of this as the plugged-in peripheral device.

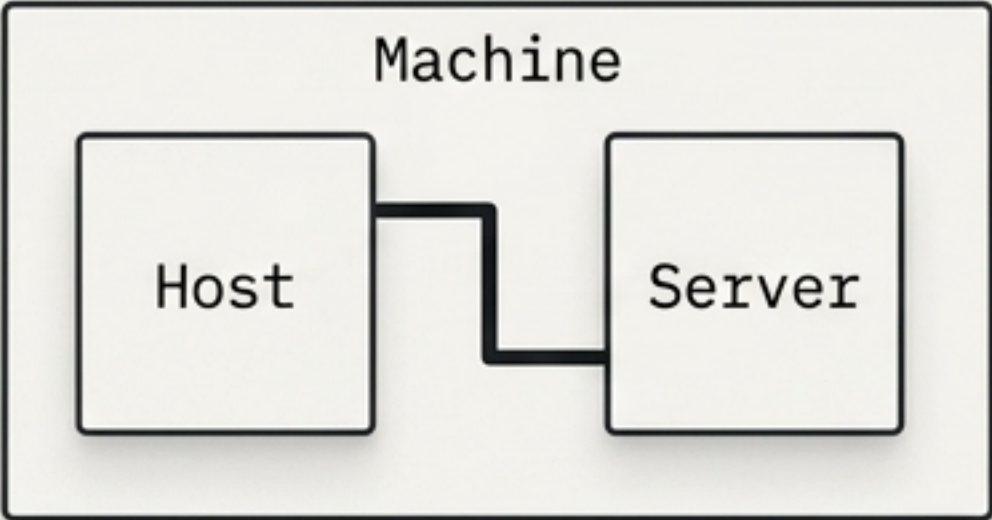
Clear Separation of Concerns: The host handles the reasoning, while the server handles the raw execution of external tasks.

Unlocking Server Capabilities



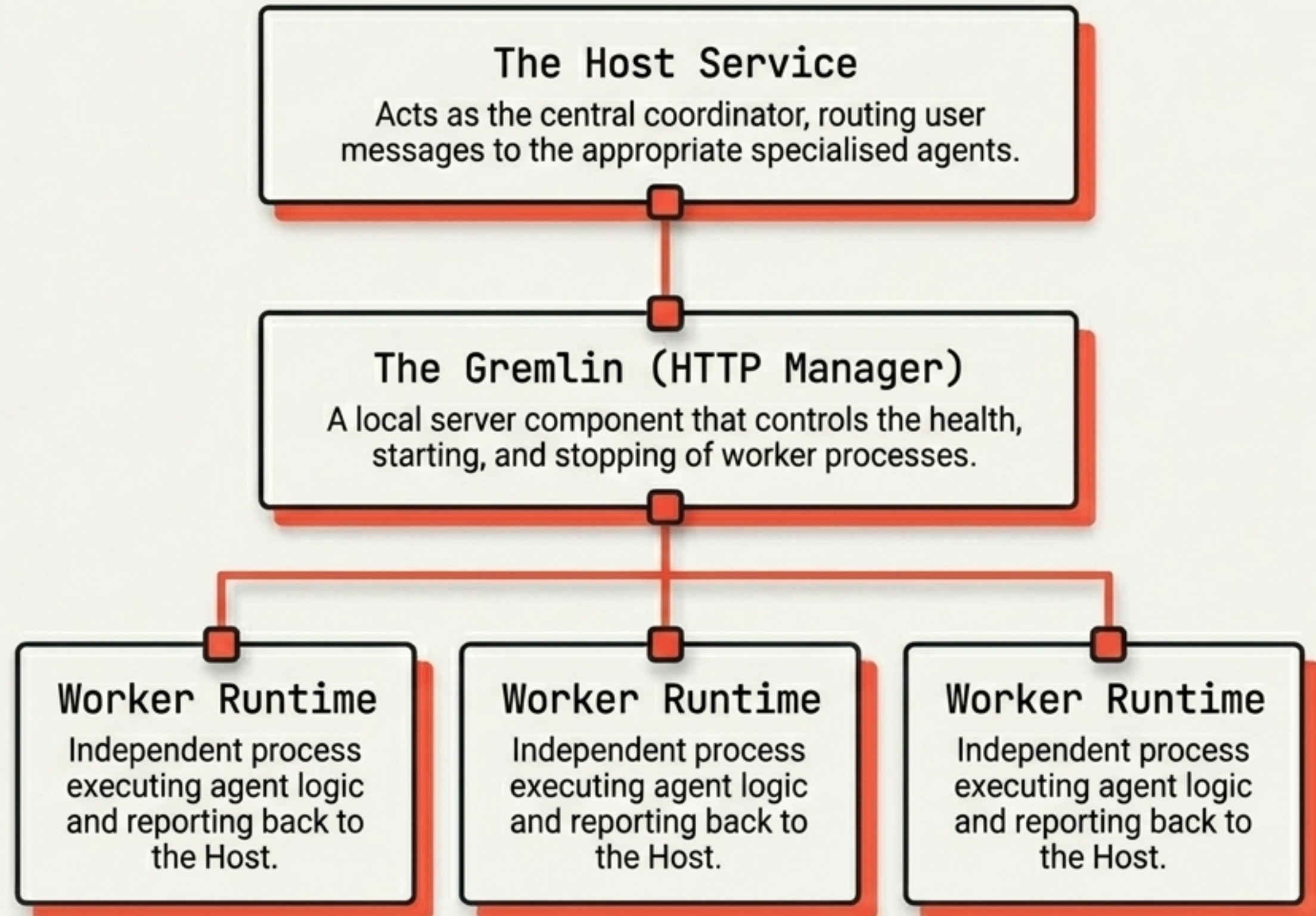
Automated Discovery: Upon connection, the client automatically queries the server and provides these capabilities directly to the agent's context.

Connecting Client and Server: Transport Mechanisms

Standard I/O (Stdio) - Local	Server-Sent Events (SSE) - Remote
 • Local Subprocess: Host spawns the MCP server as a local child process. • Direct Pipes: Communicates directly via stdin/stdout pipes. • Best Use Cases: Local tools, file system access, local code execution, or local browser automation.	 • Standalone Server: The server runs entirely independently. • HTTP Streaming: Client connects asynchronously over a persistent network connection. • Best Use Cases: Cloud APIs, remote enterprise services, and shared tool infrastructures.

Scaling with Distributed Runtimes

- **Overcoming Limits:** Running all agents in a single process quickly becomes a bottleneck as systems scale.
- **MCP as the Glue:** In this distributed pattern, every Worker Runtime functions as an independent MCP server, flawlessly orchestrated by the Host client.



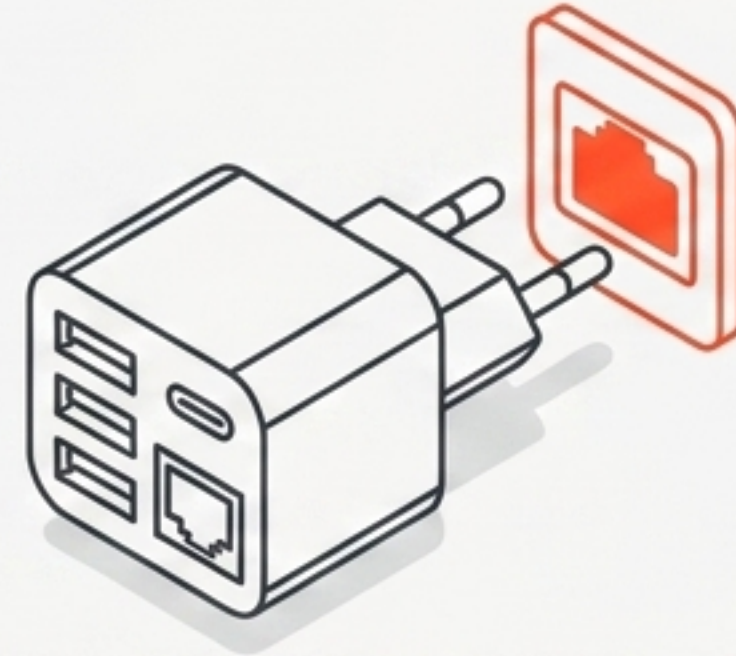
Direct Tools vs. MCP Servers

Direct @tool Functions



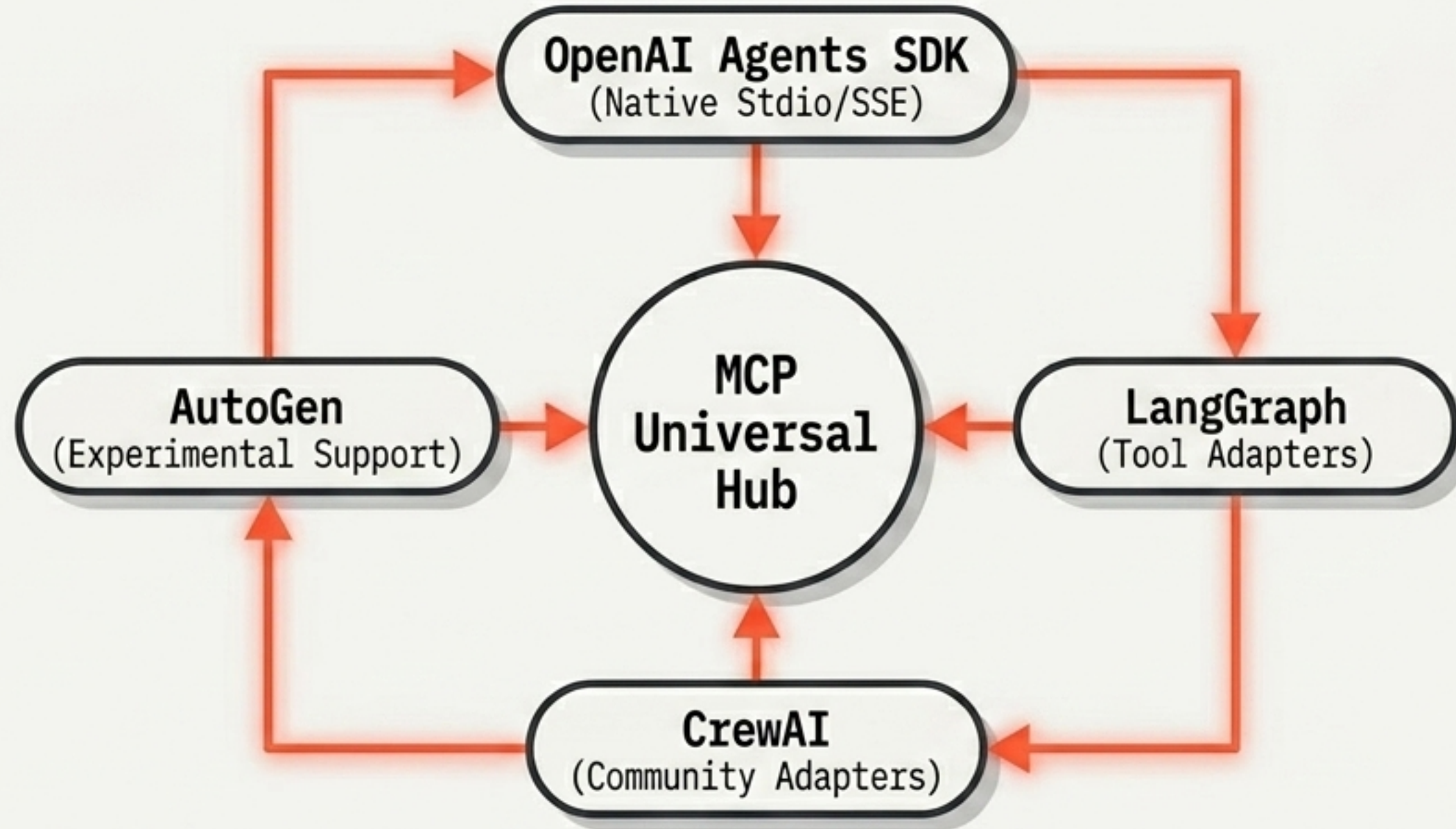
- **Definition:** Python functions living entirely inside the agent's local codebase.
- **When to Use:** Best for rapid prototyping, simple single-use logic, or highly specific team workflows.
- **Limitation:** Cannot be easily shared across different applications.

MCP Servers



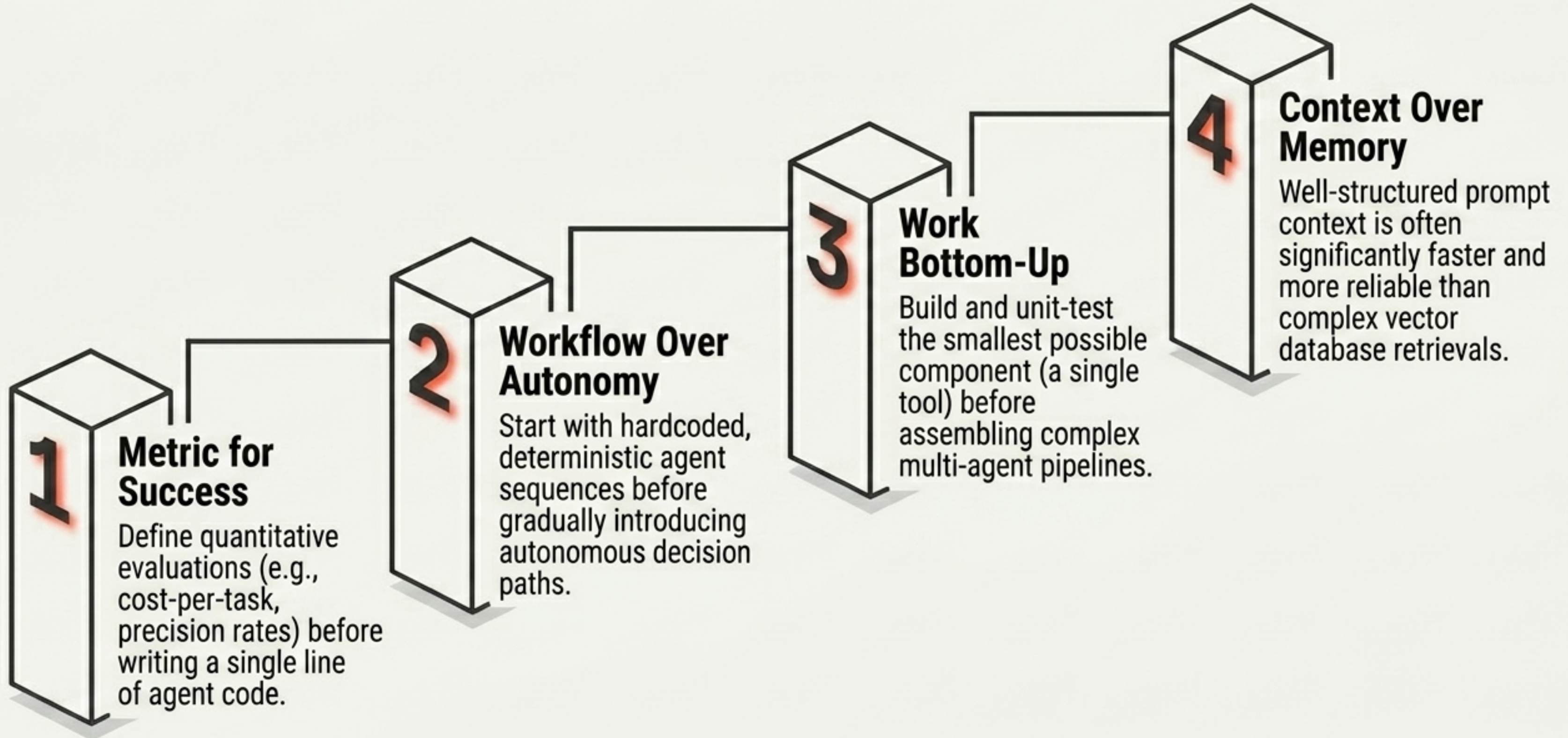
- **Definition:** Independent services exposing capabilities over the standardised protocol.
- **When to Use:** Essential for shared capabilities, cross-team libraries, and production-grade tools.
- **The Reusability Factor:** Built once, securely accessed globally by any MCP-compatible platform.

A Framework-Agnostic Ecosystem



- **Universal Compatibility:** MCP is a language and framework-agnostic standard, not proprietary software.
- **Effortless Integration:** Major frameworks connect to MCP via native layers or community-built adapters.
- **Choose by Skill, Not Hype:** The standardisation ensures the most important decision is which framework best aligns with your team's existing technology stack.

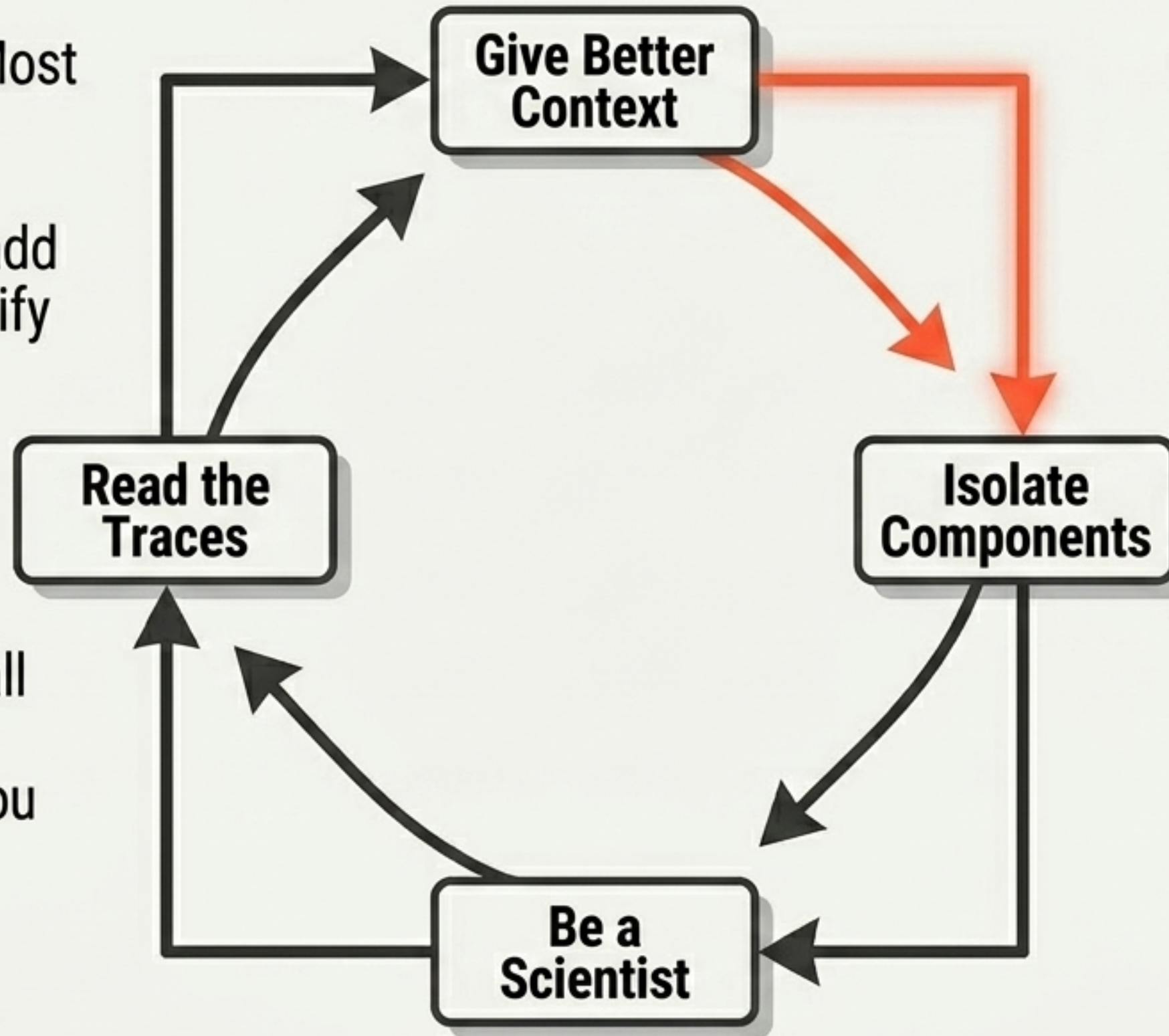
Engineering Principles for Production AI



Debugging Non-Deterministic Systems

- **Context Fixes 80%:** Most failures are prompt failures. Explicitly rewrite instructions, add constraints, and specify output formats.

- **Trace Everything:**
Use tracing tools to observe every LLM call and handoff. You cannot debug what you cannot see.



- **Test in Isolation:** Verify each tool and handoff independently using hardcoded inputs to prevent cascading failures.

- **Form Hypotheses:**
Form a specific scientific hypothesis, test it in isolation, measure the result, and iterate systematically.

Building Your First MCP Server

- **1. Initialise the Framework:** Use high-level abstractions like FastMCP to bypass complex protocol details and focus strictly on logic.
- **2. Decorate the Functions:** Apply the `@mcp.tool()` decorator to register standard Python functions into the server's capability registry.
- **3. Write Clear Docstrings:** Ensure every function has a detailed description—this is exactly what the LLM reads to decide when to use the tool.
- **4. Launch the Process:** Start the server to listen continuously on stdin/stdout for host client connections.

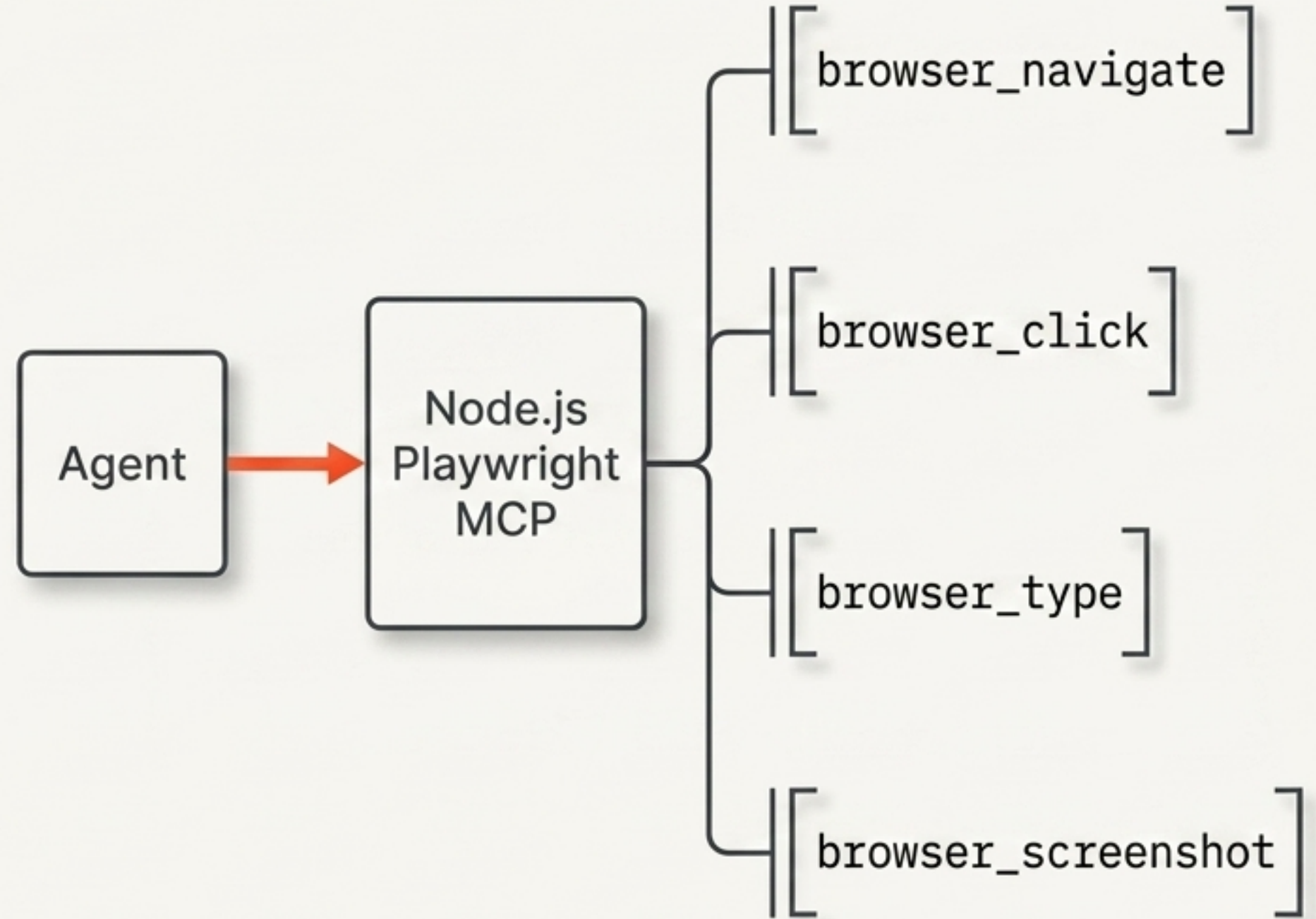
```
mcp = FastMCP('SalesMCP')

@mcp.tool()
def lookup_company(name: str) -> dict:
    """Look up company info for sales."""
    return {"revenue": "$15M ARR"}

mcp.run()
```


Instant Browser Automation via Playwright

- **Zero API Knowledge Required:** Agents gain complete web-browsing capabilities instantly by connecting to pre-built community MCP servers.
- **The Playwright Example:** Launching the `@playwright/mcp` package immediately exposes dozens of browser interaction tools.
- **Automatic Discovery:** The agent seamlessly auto-discovers and registers all navigation tools upon initial connection.
- **Visual Reasoning:** Pairing browser tools with vision models (like GPT-4o) allows agents to navigate dynamic pages using real-time screenshots.



Connecting to Cloud Services via SSE



- **Remote HTTP Streaming:** Use Server-Sent Events (SSE) to connect agents to cloud-hosted tools located anywhere in the world.
- **Secure Authentication:** Lock down connections to enterprise servers using standard HTTP bearer tokens and API keys.
- **Simultaneous Connections:** Agents can connect to multiple different MCP servers (e.g., local files and remote SaaS apps) simultaneously.
- **Shared Infrastructure:** A single enterprise MCP server can safely serve dozens of different AI applications across an entire organisation.

Synthesis: The Deep Research Agent

- **The Orchestrator:** A manager agent coordinates the workflow, delegating tasks to specialists rather than executing them directly.

- **Web Researcher:** Exclusively connected to Playwright MCP to browse live internet sources and gather raw data.

- **Structured Output:** The pipeline finishes by validating the output against a strict Pydantic schema for downstream reliability.



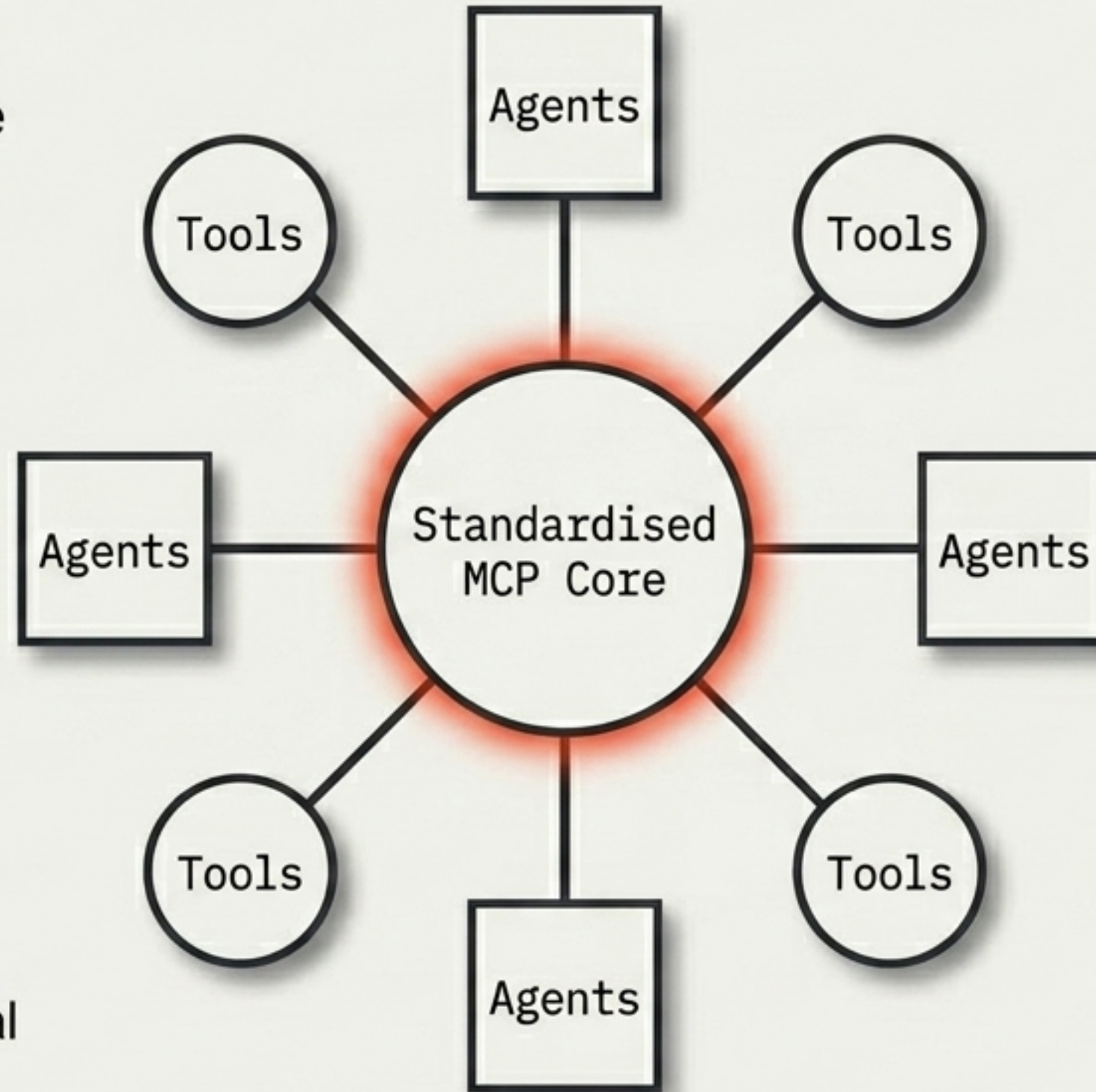
- **Report Writer:** Exclusively connected to a Data MCP server to process findings and format the document.

- **Structured Output:** The pipeline MCP, any tool or LLM in this against can be swapped the document.

- **Complete Modularity:** Thanks to MCP, any tool or LLM in this pipeline can be swapped without rewriting the orchestration.

The Future of Agent-Tool Connectivity

- **The Universal Port:** MCP acts as the USB-C of AI, solving the N×M fragmentation problem for good.



- **Build Once, Connect Everywhere:** The N+M standard ensures developers only write their tool integration once for all AI platforms.

- **Seamless Scaling:** Flexible transports (Stdio and SSE) allow seamless transitions from local prototypes to global cloud infrastructure.

- **Enterprise Readiness:** By decoupling tools from agent reasoning, MCP guarantees highly maintainable, modular, and reliable architectures.